N1

Дан ориентированный граф $G = (V, E)$ с весовой функцией $a : E \to \mathbb{R}$ и отсутствием отрицательных циклов.

Этап 1. Вычисление всех кратчайших путей

Применим алгоритм Флойда–Уоршелла для вычисления матрицы расстояний $d[i][j]$ между всеми парами вершин:

$$d[i][j] = \min \left\{ a(i,j), \min_{k \in V} \{ d[i][k] + d[k][j] \} \right\},$$

где $a(i,j) = +\infty$, если ребра (i,j) нет, и $d[i][i] = 0$ для всех i.

При этом будем поддерживать матрицу предшественников $\pi[i][j]$ для восстановления пути.

Этап 2. Поиск цикла минимального веса

Для каждой пары вершин (i,j) проверим, существует ли ребро $(j,i) \in E$. Если оно существует, то рассматриваем цикл

$$C_{ij} : i \to \cdots \to j \to i$$

с весом

$$W_{ij} = d[i][j] + a(j,i).$$

Выберем пару (i,j) такую, что W_{ij} минимально:

$$(i^*, j^*) = \arg \min_{(i,j) \text{ и } (j,i) \in E} \{ d[i][j] + a(j,i) \}.$$

Этап 3. Восстановление цикла

Используя матрицу предшественников π, восстанавливаем кратчайший путь от i^* до j^*. Далее добавляем ребро (j^*, i^*) для замыкания цикла.

Псевдокод алгоритма

Algorithm 1 Нахождение цикла минимального веса

Вход: Матрица весов $a[i][j]$ для $i, j = 1, \ldots, n$
Выход: Цикл минимального веса (список вершин) и его вес
Инициализировать $d[i][j] \leftarrow a[i][j]$ для всех i, j; $d[i][i] \leftarrow 0$
Инициализировать матрицу предшественников $\pi[i][j]$:

$$\pi[i][j] = \begin{cases} i, & \text{если } a(i,j) \neq +\infty,\ i \neq j \\ \text{undefined}, & \text{иначе} \end{cases}$$

for $k = 1$ to n **do**
  for $i = 1$ to n **do**
    for $j = 1$ to n **do**
      if $d[i][j] > d[i][k] + d[k][j]$ **then**
        $d[i][j] \leftarrow d[i][k] + d[k][j]$
        $\pi[i][j] \leftarrow \pi[k][j]$
      end if
    end for
  end for
end for
minCycle $\leftarrow +\infty$
cycle $\leftarrow$ null
for $i = 1$ to n **do**
  for $j = 1$ to n **do**
    if ребро (j, i) существует $(a(j,i) \neq +\infty)$ **then**
      $W \leftarrow d[i][j] + a(j,i)$
      if $W <$ minCycle **then**
        minCycle $\leftarrow W$
        Сохранить пару (i, j) как (i^*, j^*)
      end if
    end if
  end for
end for
if minCycle $= +\infty$ **then**
  return «Граф не содержит циклов»
else
  Восстановить путь от i^* до j^* с помощью π
  Добавить ребро (j^*, i^*) для замыкания цикла
  return найденный цикл и вес minCycle
end if

Обоснование сложности

Алгоритм Флойда–Уоршелла работает за $O(n^3)$ (строки 5–11). Далее двойной цикл (строки 12–21) также требует $O(n^2)$ операций, что не превышает асимптотической оценки $O(n^3)$. Таким образом, общая сложность алгоритма $O(n^3)$.

N2

Входные данные

Дан ориентированный граф $G = (V, E)$ с целочисленными весами ребер $w : E \to \mathbb{Z}$ и без отрицательных циклов.

Выходные данные

Цикл с суммарным весом 0 (в виде последовательности вершин), либо сообщение о его отсутствии.

Описание алгоритма

1. **Инициализация.** Добавляем вспомогательную вершину s и для каждой вершины $v \in V$ добавляем ребро (s, v) с весом 0.

2. **Вычисление кратчайших расстояний.** Запускаем алгоритм Беллмана–Форда из вершины s для вычисления расстояний $d(v)$ для всех $v \in V$. Сложность: $O(nm)$.

3. **Построение подграфа тесных рёбер.** Определяем

$$E_T = \{(u, v) \in E \mid d(v) = d(u) + w(u, v)\}.$$

   Пусть $T = (V, E_T)$.

4. **Поиск цикла.** Выполняем обход графа T (например, DFS) для поиска цикла. Если цикл найден, то он имеет суммарный вес

$$\sum_{(u,v)\in\text{цикл}} w(u, v) = 0.$$

   Если цикл не найден, сообщаем, что цикла нулевого веса в G нет.

Псевдокод

Algorithm 2 Поиск цикла с суммарным весом 0

Вход: Граф $G = (V, E)$ с целочисленными весами $w(u, v)$
Выход: Цикл с суммарным весом 0 или сообщение о его отсутствии
Добавить вспомогательную вершину: ввести s и для каждого $v \in V$ добавить ребро (s, v) с $w(s, v) = 0$
Инициализация: для всех $v \in V$ установить $d(v) = +\infty$, а $d(s) = 0$
for $i = 1$ до $|V|$ **do**
  for каждое ребро (u, v) в расширённом графе (с s) **do**
    if $d(v) > d(u) + w(u, v)$ **then**
      $d(v) \leftarrow d(u) + w(u, v)$
    end if
  end for
end for
Построить подграф $T = (V, E_T)$**:**

$$E_T \leftarrow \{(u, v) \in E \mid d(v) = d(u) + w(u, v)\}$$

Обход графа T**:** для каждой вершины $v \in V$ запустить DFS для поиска цикла
if найден цикл C **then**
  return C {Цикл имеет суммарный вес 0}
end if
return «Цикл нулевого веса не найден»

Обоснование сложности

- Алгоритм Беллмана–Форда работает за $O(nm)$.
- Построение подграфа T требует перебора всех рёбер, что даёт $O(m)$.
- Поиск цикла с помощью DFS занимает $O(n + m)$.

Таким образом, общая асимптотика алгоритма $O(nm)$.

N3

Постановка задачи

Дан ориентированный граф $G = (V, E)$ с целочисленными весами ребер $w : E \to \mathbb{Z}$. Требуется найти цикл в графе с минимальным средним весом, где средний вес цикла определяется как сумма весов его рёбер, делённая на число рёбер в цикле. Ответ необходимо найти с точностью до ε. Пусть $|w(e)| \le C$ для любого ребра e.

Основная идея алгоритма

Воспользуемся следующей редукцией: для любого λ рассмотрим модифицированные веса

$$w'(e) = w(e) - \lambda.$$

Тогда для любого цикла C имеем

$$\sum_{e \in C} w'(e) = \sum_{e \in C} \big(w(e) - \lambda\big) = \sum_{e \in C} w(e) - \lambda \cdot |C|.$$

Заметим, что $\sum_{e \in C} w(e) \le \lambda \cdot |C|$ тогда и только тогда, когда $\sum_{e \in C} w'(e) \le 0$. Таким образом, цикл с *средним весом не более* λ существует тогда и только тогда, когда в графе с модифицированными весами есть цикл с суммарным весом не более 0 (а для строго меньшего среднего веса можно требовать отрицательную сумму).

Алгоритм двоичного поиска перебирает кандидаты λ на интервале $[L, R]$, где можно взять $L = -C$ и $R = C$ (так как средний вес цикла не может выходить за эти пределы). На каждом шаге для текущего значения $\lambda = \mathrm{mid}$ строятся модифицированные веса $w'(e) = w(e) - \mathrm{mid}$ и запускается алгоритм Беллмана–Форда для проверки наличия цикла с отрицательной суммой весов. Если такой цикл находится, то минимальный средний вес не более mid и границу R можно сдвинуть вниз; иначе, граница L сдвигается вверх.

Обоснование сложности

На каждом шаге двоичного поиска выполняется проверка наличия отрицательного цикла за $O(nm)$ с помощью алгоритма Беллмана–Форда. Число итераций двоичного поиска составляет $O\left(\log \frac{C}{\varepsilon}\right)$. Таким образом, итоговая асимптотика алгоритма равна

$$O\Big(nm \cdot \log \frac{C}{\varepsilon}\Big).$$

N4

Вход: неориентированный граф $G = (V, E)$ с весами $w(e)$, начальная вершина s и конечная вершина t.

Выход: Минимальная стоимость пути (по определению, максимум весов на пути) от s до t.

Algorithm 3 Bottleneck-Dijkstra

Инициализация: для всех $v \in V$ установить $d(v) \leftarrow +\infty$; $d(s) \leftarrow 0$.
Поместить пару $(s, 0)$ в приоритетную очередь Q с ключом 0.
while Q не пуста **do**
  Извлечь $(u, d(u))$ с минимальным значением из Q.
  if $u = t$ **then**
    return $d(t)$.
  end if
  for каждого ребра (u, v) с весом $w(u, v)$ **do**
    $d' \leftarrow \max\{d(u),\, w(u, v)\}$.
    if $d' < d(v)$ **then**
      $d(v) \leftarrow d'$.
      Обновить/вставить $(v, d(v))$ в Q.
    end if
  end for
end while
return $d(t)$.

N5

Постановка задачи

Даны переменные $x_1, \ldots, x_n$ и m неравенств двух типов:

1. $x_i \le c$,
2. $x_i \le x_j + c$,

где i, j и c могут различаться для каждого неравенства. Требуется найти набор значений $x_1, \ldots, x_n$, на котором достигается максимум

$$S = x_1 + x_2 + \cdots + x_n,$$

либо сообщить, что $S = +\infty$. Сложность алгоритма должна быть $O(nm)$.

Описание алгоритма

1. **Построение графа.** Вводим вспомогательную вершину s с зафиксированным значением $x_s = 0$. Для каждого неравенства:

   - Если задано $x_i \le c$, добавляем ребро (s, i) с весом c.
   - Если задано $x_i \le x_j + c$, добавляем ребро (j, i) с весом c.

   Тогда каждое ребро (u, v) соответствует ограничению

   $$x_v \le x_u + w(u, v).$$

2. **Поиск длиннейших путей.** Инициализируем $x(s) = 0$ и для всех i положим $x(i) = -\infty$. Выполняем n итераций релаксаций:

   $$\text{для каждого ребра } (u, v): \quad x(v) \leftarrow \max\{x(v),\, x(u) + w(u, v)\}.$$

   Если на n-ой итерации происходит улучшение, то существует положительный цикл, достижимый из s, и максимум равен $+\infty$.

3. **Ответ.** Если положительный цикл не найден, то оптимальные значения переменных равны найденным $x(1), \ldots, x(n)$, а максимум суммы равен

   $$S^* = \sum_{i=1}^{n} x(i).$$

Обоснование сложности

На каждой из n итераций перебирается $O(m)$ ребер, что дает общую сложность $O(nm)$.

N6

Постановка задачи

Дан связный взвешенный граф $G = (V, E)$ с n вершинами и m рёбрами. Требуется за $O(n + m)$ найти некоторое множество рёбер $\mathcal{E} \subseteq E$ размера $\Theta(n)$, такое что существует минимальное остовное дерево T графа G, для которого $\mathcal{E} \subseteq T$.

Идея решения

Используем первую фазу алгоритма Борувки. На начальном этапе каждая вершина рассматривается как отдельная компонента. Для каждой компоненты C выбирается ребро минимального веса, исходящее из C (то есть такое ребро, что один конец принадлежит C, а другой — нет). По свойству Борувки такое ребро является безопасным, то есть принадлежит некоторому MST. Объединяя все выбранные рёбра, получаем множество $\mathcal{E}$ рёбер, которое можно дополнить до минимального остова.

Псевдокод

Algorithm 4 Нахождение множества рёбер, расширяемых до MST

Вход: Граф $G = (V, E)$ с весами $w(e)$
Выход: Множество рёбер $\mathcal{E} \subseteq E$, которое можно дополнить до MST
Инициализировать множество рёбер $\mathcal{E} \leftarrow \varnothing$
Для каждой вершины $v \in V$ создать отдельную компоненту (используя структуру Union-Find)
for каждой компоненты C (на начальном этапе это каждая вершина) **do**
    Инициализировать $e_C \leftarrow$ null
    for каждого ребра $e = (u, v)$ инцидентного компоненте C **do**
        if $u \in C$ и $v \notin C$ (или наоборот) **then**
            if e_C не определено **или** $w(e) < w(e_C)$ **then**
                $e_C \leftarrow e$
            end if
        end if
    end for
    if e_C найдено **then**
        Добавить e_C в $\mathcal{E}$
    end if
end for
return $\mathcal{E}$

Обоснование сложности

На начальном этапе каждая вершина является отдельной компонентой, и для каждой компоненты мы просматриваем все инцидентные рёбра. Общее число операций пропорционально $O(n + m)$. Таким образом,

алгоритм работает за $O(n + m)$.